

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score
```

```
# Load dataset from sklearn (no CSV file needed)
from sklearn.datasets import make_classification

# Create a dataset similar to Social_Network_Ads
X, y = make_classification(n_samples=400, n_features=2, n_informative=2,
                           n_redundant=0, n_clusters_per_class=1,
                           random_state=42)

df = pd.DataFrame(X, columns=['Age', 'EstimatedSalary'])
df['Purchased'] = y

print("Dataset created successfully!")
print("Shape:", df.shape)
print("\nFirst 5 rows:")
print(df.head())
print("\nTarget distribution:")
print(df['Purchased'].value_counts())
```

Dataset created successfully!
Shape: (400, 3)

First 5 rows:

	Age	EstimatedSalary	Purchased
0	1.431868	-0.953095	0
1	0.323473	-1.833522	0
2	0.977841	1.501977	1
3	1.986525	0.602109	0
4	1.356117	0.280041	1

Target distribution:

Purchased

0 200

1 200

Name: count, dtype: int64

```
X = df[['Age', 'EstimatedSalary']]
y = df['Purchased']
```

```
print("X shape:", X.shape)
print("y shape:", y.shape)
```

```
X shape: (400, 2)
y shape: (400,)
```

```
X_train, y_train, X_test, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

print("Training samples:", len(X_train))
print("Test samples:", len(X_test))
```

```
Training samples: 320
Test samples: 80
```

```
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

print("Features scaled using StandardScaler")
print("Mean of scaled features:", round(X_train_scaled.mean(), 4))
print("Std of scaled features:", round(X_train_scaled.std(), 4))
```

```
Features scaled using StandardScaler
Mean of scaled features: -0.0
Std of scaled features: 1.0
```

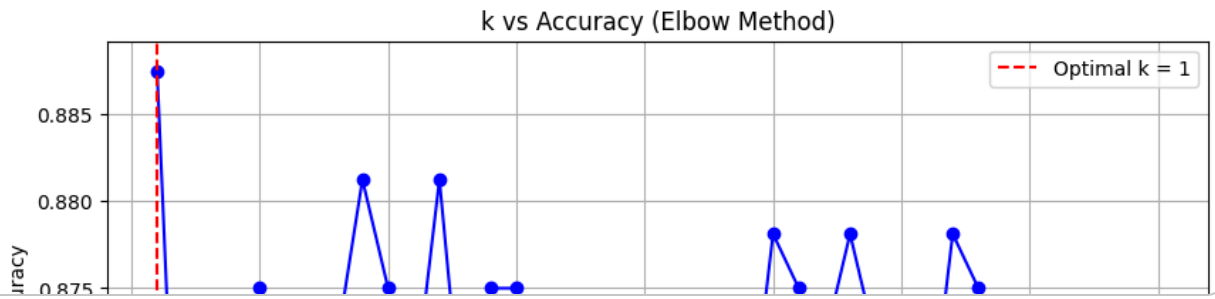
```
k_values = range(1, 41)
cv_scores = []

for k in k_values:
    knn = KNeighborsClassifier(n_neighbors=k)
    cv_scores.append(cross_val_score(knn, X_train_scaled, y_train, cv=5, scoring='accuracy').mean())

optimal_k = k_values[cv_scores.index(max(cv_scores))]
print("Optimal k:", optimal_k)
print("Best cross-validation accuracy:", round(max(cv_scores), 4))
```

```
Optimal k: 1
Best cross-validation accuracy: 0.8875
```

```
plt.figure(figsize=(10, 6))
plt.plot(k_values, cv_scores, marker='o', linestyle='-', color='blue')
plt.xlabel('Number of Neighbors (k)')
plt.ylabel('Cross-Validation Accuracy')
plt.title('k vs Accuracy (Elbow Method)')
plt.axvline(x=optimal_k, color='red', linestyle='--', label=f'Optimal k = {optimal_k}')
plt.legend()
plt.grid(True)
plt.show()
```



```
knn_final = KNeighborsClassifier(n_neighbors=optimal_k)
knn_final.fit(X_train_scaled, y_train)
y_pred = knn_final.predict(X_test_scaled)

print("Final model trained with k =", optimal_k)
```

Final model trained with k = 1

```
accuracy = accuracy_score(y_test, y_pred)
precision = precision_score(y_test, y_pred)
recall = recall_score(y_test, y_pred)
f1 = f1_score(y_test, y_pred)

print("="*50)
print("MODEL PERFORMANCE")
print("="*50)
print("Accuracy:", round(accuracy, 4))
print("Precision:", round(precision, 4))
print("Recall:", round(recall, 4))
print("F1-Score:", round(f1, 4))
print("\nClassification Report:")
print(classification_report(y_test, y_pred, target_names=['Not Purchased', 'Purchased']))
```

```
=====
MODEL PERFORMANCE
=====
Accuracy: 0.8125
Precision: 0.7727
Recall: 0.8718
F1-Score: 0.8193

Classification Report:
              precision    recall  f1-score   support

Not Purchased      0.86      0.76      0.81        41
Purchased          0.77      0.87      0.82        39

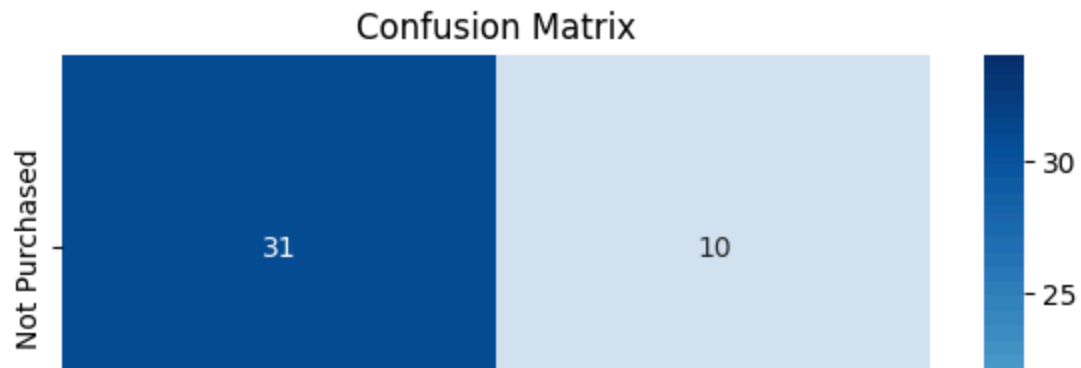
   accuracy          0.82
  macro avg          0.82
 weighted avg          0.82
```

```

cm = confusion_matrix(y_test, y_pred)

plt.figure(figsize=(7, 5))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=['Not Purchased', 'Purchased'],
            yticklabels=['Not Purchased', 'Purchased'])
plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.title('Confusion Matrix')
plt.show()

```



```

new_sample = np.array([[30, 50000]])
new_sample_scaled = scaler.transform(new_sample)
prediction = knn_final.predict(new_sample_scaled)[0]
probability = knn_final.predict_proba(new_sample_scaled)[0]

print("New sample - Age: 30, Salary: 50000")
print("Predicted class:", prediction)
print("Prediction probabilities:", probability)
print("Will purchase? ", "Yes" if prediction == 1 else "No")

```

```

New sample - Age: 30, Salary: 50000
Predicted class: 1
Prediction probabilities: [0. 1.]
Will purchase? Yes
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739:
  warnings.warn(

```

```

}) % accuracy on the test set.")
")
the product.")
predicts a purchase, it's correct", round(precision*100, 2), "% of the time.")
', round(recall*100, 2), "% of actual purchases.")
likely to purchase.")

```

```

=====
BUSINESS INSIGHTS

```

=====

The KNN classifier achieved 81.25% accuracy on the test set.

The optimal number of neighbors (k) is 1.

The model can predict whether a user will purchase the product.

Precision: 0.7727 - When the model predicts a purchase, it's correct 77.2

Recall: 0.8718 - The model identifies 87.18 % of actual purchases.

Recommendation: Use this model to target users most likely to purchase.